

4 ASES

1. Objetivo general del sistema

Queremos construir una API para una casa de apuestas donde el usuario pueda:

- ❖ Ver deportes disponibles
- ❖ Ver eventos deportivos activos
- ❖ Ver cuotas actualizadas
- ❖ Apostar sobre una cuota
- ❖ Ver su historial de apuestas
- ❖ Ver saldo disponible
- ❖ Recibir cambios en tiempo real
- ❖ No recargar la página manualmente

Y el administrador pueda:

- ❖ Gestionar deportes activos
- ❖ Ver eventos sincronizados
- ❖ Ver consumo de The Odds API
- ❖ Ver apuestas realizadas
- ❖ Liquidar apuestas
- ❖ Controlar usuarios
- ❖ Controlar wallets
- ❖ Configurar frecuencia de actualización
- ❖ Configurar mercados y regiones

2. Stack recomendado

Para este proyecto usaría:

Backend: Laravel 12

Arquitectura: Monolito modular

Base de datos: MySQL

Cache: Redis

Tiempo real: Laravel Reverb

Colas: Laravel Queues con Redis

Scheduler: Laravel Task Scheduling

Auth: Laravel Sanctum

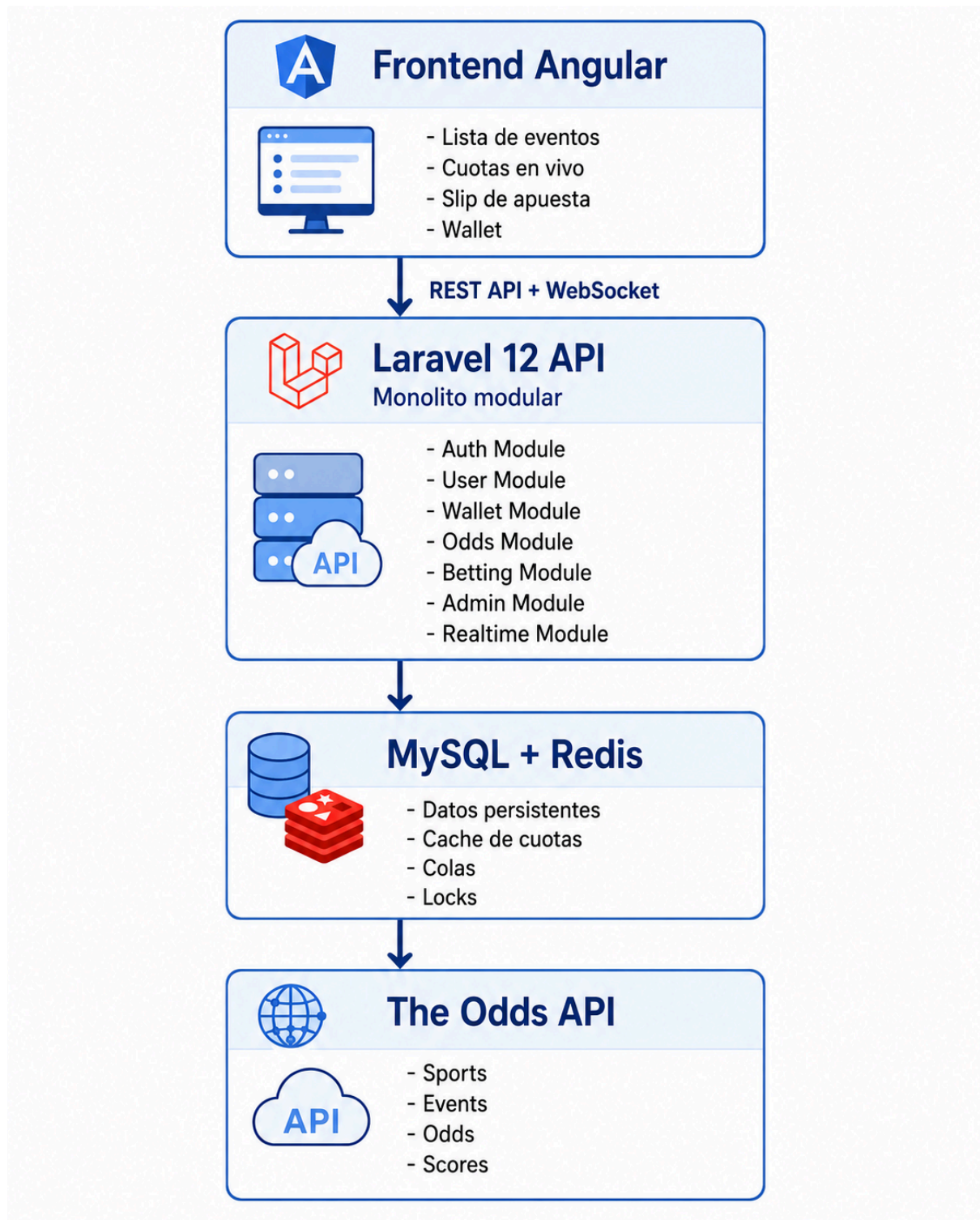
Roles/permisos: Spatie Permissions

Frontend: Angular o Ionic

Cliente WebSocket: Laravel Echo

Proveedor externo: The Odds API .

3. Arquitectura general



4. Módulos del monolito

La estructura sería modular, pero dentro de un solo Laravel.

app/

- └─ Modules/
 - └─ Odds/
 - └─ Actions/
 - | └─ SyncSportsAction.php
 - | └─ SyncEventsAction.php
 - | └─ SyncOddsAction.php
 - | └─ StoreOddsSnapshotAction.php
 - └─ Controllers/
 - | └─ SportController.php
 - | └─ EventController.php
 - | └─ OddsController.php
 - └─ DTOs/
 - | └─ OddsQueryData.php
 - | └─ OddsSnapshotData.php
 - └─ Events/
 - | └─ OddsUpdated.php
 - | └─ EventStatusUpdated.php
 - └─ Jobs/
 - | └─ SyncSportsJob.php
 - | └─ SyncEventsJob.php
 - | └─ SyncOddsJob.php
 - └─ Models/
 - | └─ Sport.php

```
| |— SportEvent.php
| |— Bookmaker.php
| |— OddsSnapshot.php
|— Requests/
| |— OddsQueryRequest.php
|— Resources/
| |— SportResource.php
| |— SportEventResource.php
| |— OddsResource.php
|— Services/
| |— OddsApiClient.php
| |— OddsApiService.php
| |— OddsUsageService.php
```

Historias de usuario — Casa de apuestas

Módulo 1: Autenticación y usuarios

HU-01: Registro de usuario

Como usuario visitante,
quiero registrarme en la plataforma,
para poder acceder al sistema, tener una cuenta y realizar apuestas.

Criterios de aceptación

- ❖ El usuario debe ingresar nombre, correo y contraseña.
- ❖ El correo debe ser único.
- ❖ La contraseña debe cumplir reglas mínimas de seguridad.
- ❖ Al registrarse, el usuario debe recibir el rol customer.
- ❖ Al registrarse, se debe crear automáticamente una wallet asociada.
- ❖ El sistema debe devolver un token de autenticación o permitir login posterior.

Prioridad: **ALTA**

HU-02: Inicio de sesión

Como usuario registrado,
quiero iniciar sesión con mi correo y contraseña,
para acceder a mi cuenta y utilizar las funciones de apuestas.

Criterios de aceptación

- ❖ El usuario debe ingresar correo y contraseña.
- ❖ Si las credenciales son correctas, el sistema devuelve un token Sanctum.
- ❖ Si las credenciales son incorrectas, el sistema devuelve un mensaje claro.
- ❖ El usuario bloqueado o suspendido no debe poder iniciar sesión.

Prioridad: **ALTA**

HU-03: Cerrar sesión

Como usuario autenticado,
quiero cerrar sesión,
para finalizar mi acceso de forma segura.

Criterios de aceptación

- ❖ El sistema debe revocar el token actual.
- ❖ El usuario no debe poder consumir endpoints protegidos después del logout.
- ❖ La respuesta debe confirmar el cierre de sesión.

Prioridad: **ALTA**

HU-04: Consultar perfil del usuario

Como usuario autenticado,
quiero consultar mi perfil,
para verificar mis datos personales y estado de cuenta.

Criterios de aceptación

- ❖ El sistema debe devolver datos básicos del usuario.
- ❖ Debe incluir rol, estado de la cuenta y fecha de registro.
- ❖ No debe exponer datos sensibles como contraseña.

Prioridad: **MEDIA**

HU-05: Bloqueo de usuario por administrador

Como administrador,
quiero bloquear o suspender usuarios,
para impedir que cuentas sospechosas sigan apostando.

Criterios de aceptación

- ❖ Solo el administrador puede bloquear usuarios.
- ❖ Un usuario bloqueado no puede iniciar sesión.
- ❖ Un usuario bloqueado no puede crear apuestas.
- ❖ El sistema debe guardar la fecha y motivo del bloqueo.

Prioridad: **ALTA**

Módulo 2: Deportes y eventos deportivos

HU-06: Sincronizar deportes desde The Odds API

Como administrador,
quiero sincronizar los deportes disponibles desde The Odds API,
para mantener actualizado el catálogo de deportes de la plataforma.

Criterios de aceptación

- ❖ El sistema debe consultar el endpoint de deportes de The Odds API.
- ❖ Los deportes deben guardarse o actualizarse en la base de datos.
- ❖ No se deben duplicar deportes existentes.
- ❖ Se debe guardar sport_key, título, grupo y estado activo.
- ❖ El sistema debe registrar la sincronización.

Prioridad: **ALTA**

HU-07: Listar deportes activos

Como usuario,
quiero ver los deportes disponibles,
para seleccionar en cuál quiero revisar eventos y cuotas.

Criterios de aceptación

- ❖ El sistema debe devolver solo deportes activos.
- ❖ La respuesta debe incluir nombre, clave del deporte y grupo.

- ❖ La lista debe poder ser consumida por el frontend.

Prioridad: **ALTA**

HU-08: Activar o desactivar deporte

Como administrador,
quiero activar o desactivar deportes,
para controlar qué deportes estarán disponibles para apostar.

Criterios de aceptación

- ❖ Solo el administrador puede cambiar el estado de un deporte.
- ❖ Si un deporte está desactivado, no debe mostrarse al usuario final.
- ❖ No se deben permitir nuevas apuestas en deportes desactivados.

Prioridad: **MEDIA**

HU-09: Sincronizar eventos deportivos

Como sistema,
quiero sincronizar eventos deportivos desde The Odds API,
para mantener actualizados los partidos disponibles.

Criterios de aceptación

- ❖ El sistema debe consultar eventos por deporte activo.
- ❖ Los eventos deben guardarse con `external_event_id`.
- ❖ Se debe guardar equipo local, visitante, deporte y fecha de inicio.
- ❖ Si el evento ya existe, se debe actualizar.
- ❖ No debe haber eventos duplicados.

Prioridad: **ALTA**

HU-10: Listar eventos por deporte

Como usuario,
quiero ver los eventos disponibles de un deporte,
para elegir un partido en el cual apostar.

Criterios de aceptación

- ❖ El usuario puede filtrar eventos por sport_key.
- ❖ El sistema debe mostrar eventos activos y disponibles.
- ❖ Cada evento debe incluir equipos, fecha, estado y deporte.
- ❖ Los eventos cerrados o cancelados no deben permitir apuestas.

Prioridad: **ALTA**

HU-11: Ver detalle de un evento

Como usuario,
quiero ver el detalle de un evento deportivo,
para consultar información del partido antes de apostar.

Criterios de aceptación

- ❖ El sistema debe mostrar equipos, fecha, estado y cuotas disponibles.
- ❖ Si el evento está cerrado, debe indicarlo claramente.
- ❖ Si el evento está en vivo, debe mostrar una etiqueta "En vivo".

Prioridad: **ALTA**

HU-12: Actualizar estado de eventos

Como sistema,
quiero actualizar el estado de los eventos deportivos,
para saber si están programados, en vivo, finalizados o cancelados.

Criterios de aceptación

- ❖ El sistema debe actualizar estados automáticamente.
- ❖ Los estados mínimos son scheduled, live, completed, cancelled.
- ❖ Un evento completed no debe aceptar nuevas apuestas.
- ❖ Un evento cancelled debe permitir devolución de apuestas pendientes.

Prioridad: **ALTA**

Módulo 3: Cuotas y proveedor externo

HU-13: Consultar cuotas desde The Odds API

Como sistema,
quiero consultar cuotas desde The Odds API,
para ofrecer mercados actualizados a los usuarios.

Criterios de aceptación

- ❖ El sistema debe consultar cuotas por deporte, región y mercado.
- ❖ La API key debe estar protegida en el backend.
- ❖ El frontend nunca debe conocer la API key.
- ❖ El sistema debe manejar errores de conexión.
- ❖ El sistema debe registrar cada consulta realizada.

Prioridad: **ALTA**

HU-14: Guardar snapshots de cuotas

Como sistema,
quiero guardar snapshots de cuotas,
para conservar el valor exacto de una cuota en un momento específico.

Criterios de aceptación

- ❖ Cada cuota consultada debe guardarse como snapshot.
- ❖ El snapshot debe incluir evento, mercado, selección, bookmaker y precio.
- ❖ El sistema debe guardar fecha y hora del snapshot.
- ❖ Si un usuario apuesta, la apuesta debe quedar ligada a un snapshot.

Prioridad: **ALTA**

HU-15: Detectar cambios de cuotas

Como sistema,
quiero detectar cuando una cuota cambia,
para actualizar al usuario en tiempo real.

Criterios de aceptación

- ❖ El sistema debe comparar la cuota nueva con la cuota anterior.
- ❖ Si el precio cambia, debe guardar el nuevo snapshot.
- ❖ Si no hay cambios, no debe emitir eventos innecesarios.
- ❖ El sistema puede usar un hash para detectar cambios.

Prioridad: **ALTA**

HU-16: Listar cuotas disponibles

Como usuario,
quiero ver las cuotas disponibles de un evento,
para elegir una selección y realizar una apuesta.

Criterios de aceptación

- ❖ El sistema debe devolver cuotas agrupadas por evento y mercado.
- ❖ Cada cuota debe incluir selección, precio, bookmaker y vigencia.
- ❖ Las cuotas suspendidas no deben permitir apuesta.
- ❖ Las cuotas vencidas deben ocultarse o mostrarse como no disponibles.

Prioridad: **ALTA**

HU-17: Controlar consumo de The Odds API

Como administrador,
quiero ver el consumo de créditos de The Odds API,
para controlar el uso mensual del proveedor.

Criterios de aceptación

- ❖ El sistema debe guardar créditos usados por consulta.
- ❖ El sistema debe guardar requests restantes si el proveedor lo devuelve.
- ❖ El admin debe poder ver historial de consumo.
- ❖ El sistema debe evitar llamadas innecesarias.

Prioridad: **MEDIA**

HU-18: Configurar región, mercado y frecuencia de sincronización

Como administrador,
quiero configurar región, mercado y frecuencia de actualización,
para controlar el consumo y comportamiento de las cuotas.

Criterios de aceptación

- ❖ El admin puede definir región por defecto.
- ❖ El admin puede definir mercado por defecto.
- ❖ El admin puede configurar intervalo de sincronización.
- ❖ Los cambios deben aplicarse sin modificar código.

Prioridad: **MEDIA**

Módulo 4: Tiempo real

HU-19: Actualizar cuotas en tiempo real

Como usuario,
quiero que las cuotas se actualicen automáticamente,
para no tener que recargar la página.

Criterios de aceptación

- ❖ El frontend debe recibir cambios mediante WebSockets.
- ❖ Cuando una cuota cambia, la interfaz debe actualizar el valor.
- ❖ El usuario debe ver visualmente si una cuota subió o bajó.
- ❖ No debe ser necesario recargar la página.

Prioridad: **ALTA**

HU-20: Notificar eventos cerrados en tiempo real

Como usuario,
quiero saber cuando un evento se cierra o suspende,
para evitar intentar apostar en eventos no disponibles.

Criterios de aceptación

- ❖ Cuando un evento cambia a completed, cancelled o suspended, se debe emitir evento realtime.
- ❖ El frontend debe bloquear botones de apuesta.
- ❖ El usuario debe ver un mensaje claro.

Prioridad: **ALTA**

HU-21: Actualizar wallet en tiempo real

Como usuario,
quiero que mi saldo se actualice automáticamente,
para ver el impacto inmediato de apuestas, depósitos o ganancias.

Criterios de aceptación

- ❖ Cuando se cree una transacción de wallet, se debe emitir WalletUpdated.
- ❖ El usuario debe ver su saldo actualizado sin recargar.
- ❖ Solo el usuario dueño de la wallet debe recibir ese evento.

Prioridad: **ALTA**

HU-22: Actualizar estado de apuestas en tiempo real

Como usuario,
quiero recibir cambios del estado de mis apuestas,
para saber si fueron aceptadas, ganadas, perdidas o canceladas.

Criterios de aceptación

- ❖ El sistema debe emitir BetStatusUpdated.
- ❖ El evento debe llegar solo al dueño de la apuesta.
- ❖ La interfaz debe actualizar el historial automáticamente.

Prioridad: **ALTA**

HU-23: Dashboard admin en tiempo real

Como administrador,
quiero ver métricas del sistema en tiempo real,
para monitorear apuestas, consumo y actividad.

Criterios de aceptación

- ❖ El dashboard debe actualizar apuestas recientes.
- ❖ Debe mostrar cambios de consumo de API.
- ❖ Debe mostrar movimientos importantes de wallet.
- ❖ Solo admins pueden escuchar el canal admin.

Prioridad: **MEDIA**

Módulo 5: Wallet

HU-24: Crear wallet automáticamente

Como sistema,
quiero crear una wallet cuando se registra un usuario,
para que pueda manejar saldo dentro de la plataforma.

Criterios de aceptación

- ❖ Cada usuario debe tener una sola wallet.
- ❖ La wallet inicia con saldo 0.
- ❖ La wallet debe tener moneda definida, por ejemplo USD.
- ❖ No debe duplicarse una wallet para el mismo usuario.

Prioridad: **ALTA**

HU-25: Consultar saldo

Como usuario,
quiero consultar mi saldo disponible,
para saber cuánto puedo apostar.

Criterios de aceptación

- ❖ El sistema debe devolver balance y locked_balance.
- ❖ El usuario solo puede ver su propia wallet.
- ❖ El saldo debe mostrarse actualizado.

Prioridad: **ALTA**

HU-26: Registrar depósito manual

Como administrador,
quiero registrar depósitos manuales,
para aumentar el saldo de un usuario cuando corresponda.

Criterios de aceptación

- ❖ Solo el administrador puede registrar depósitos.
- ❖ El monto debe ser positivo.
- ❖ Se debe crear una wallet_transaction.
- ❖ El saldo del usuario debe actualizarse.
- ❖ Debe emitirse WalletUpdated.

Prioridad: **ALTA**

HU-27: Registrar retiro manual

Como administrador,
quiero registrar retiros manuales,
para descontar saldo cuando el usuario solicite retirar dinero.

Criterios de aceptación

- ❖ Solo el administrador puede registrar retiros.
- ❖ El usuario debe tener saldo suficiente.
- ❖ Se debe crear una wallet_transaction.
- ❖ El saldo debe actualizarse.
- ❖ Debe emitirse WalletUpdated.

Prioridad: **MEDIA**

HU-28: Bloquear saldo al apostar

Como sistema,
quiero bloquear el monto apostado,
para asegurar que el usuario no use el mismo saldo en otra apuesta.

Criterios de aceptación

- ❖ Al crear una apuesta, el monto pasa de balance a locked_balance.
- ❖ El proceso debe ejecutarse dentro de una transacción de base de datos.
- ❖ Si la apuesta falla, el saldo no debe modificarse.
- ❖ Se debe crear una transacción tipo bet_hold.

Prioridad: **ALTA**

HU-29: Liberar o pagar saldo al liquidar apuesta

Como sistema,
quiero actualizar el saldo al liquidar una apuesta,
para reflejar pérdidas, ganancias o devoluciones.

Criterios de aceptación

- ❖ Si la apuesta gana, se libera locked_balance y se suma la ganancia.
- ❖ Si la apuesta pierde, se descuenta el locked_balance.
- ❖ Si se cancela, se devuelve el monto apostado.
- ❖ Toda operación debe generar wallet_transaction.
- ❖ Debe emitirse WalletUpdated.

Prioridad: **ALTA**

Módulo 6: Apuestas

HU-30: Crear apuesta simple

Como usuario autenticado,
quiero crear una apuesta simple,
para participar en un evento deportivo.

Criterios de aceptación

- ❖ El usuario debe estar autenticado.
- ❖ El usuario debe enviar evento, mercado, selección y monto.
- ❖ El sistema debe obtener la cuota desde el snapshot del backend.
- ❖ No se debe confiar en la cuota enviada por el frontend.
- ❖ El usuario debe tener saldo suficiente.
- ❖ El evento debe estar activo.
- ❖ La apuesta debe quedar en estado pending o accepted.

Prioridad: **ALTA**

HU-31: Calcular ganancia potencial

Como usuario,
quiero ver la ganancia potencial antes de apostar,
para decidir si confirmo la apuesta.

Criterios de aceptación

- ❖ El sistema debe calcular $\text{monto} \times \text{cuota}$.
- ❖ El cálculo debe usar la cuota vigente del backend.
- ❖ Si la cuota cambió, debe informarse al usuario.

Prioridad: **ALTA**

HU-32: Confirmar cambio de cuota antes de apostar

Como usuario,
quiero ser notificado si una cuota cambia antes de confirmar mi apuesta,
para decidir si acepto el nuevo valor.

Criterios de aceptación

- ❖ Si la cuota seleccionada cambió, el sistema debe rechazar la apuesta inicial.
- ❖ La respuesta debe incluir la cuota anterior y la nueva.
- ❖ El frontend debe solicitar confirmación.
- ❖ Solo si el usuario confirma, se crea la apuesta con la nueva cuota.

Prioridad: **ALTA**

HU-33: Validar monto mínimo y máximo de apuesta

Como administrador,
quiero definir montos mínimos y máximos de apuesta,
para controlar el riesgo de la plataforma.

Criterios de aceptación

- ❖ El sistema debe validar monto mínimo.
- ❖ El sistema debe validar monto máximo.
- ❖ Los límites deben ser configurables.
- ❖ Si el monto no cumple, la apuesta debe rechazarse.

Prioridad: **ALTA**

HU-34: Consultar historial de apuestas

Como usuario,
quiero consultar mi historial de apuestas,
para revisar mis apuestas pendientes, ganadas o perdidas.

Criterios de aceptación

- ❖ El usuario solo puede ver sus propias apuestas.
- ❖ El historial debe estar paginado.
- ❖ Se debe poder filtrar por estado.
- ❖ Cada apuesta debe mostrar monto, cuota, posible ganancia y estado.

Prioridad: **ALTA**

HU-35: Ver detalle de una apuesta

Como usuario,
quiero ver el detalle de una apuesta,
para conocer evento, selección, cuota y estado.

Criterios de aceptación

- ❖ El sistema debe mostrar datos completos de la apuesta.
- ❖ Debe mostrar snapshot usado.
- ❖ Debe mostrar transacciones asociadas.
- ❖ El usuario solo puede ver apuestas propias.

Prioridad: **MEDIA**

HU-36: Cancelar apuesta pendiente

Como usuario,
quiero cancelar una apuesta pendiente cuando sea permitido,
para recuperar mi saldo si aún no ha sido aceptada o procesada.

Criterios de aceptación

- ❖ Solo se pueden cancelar apuestas en estado pending.
- ❖ Si se cancela, el saldo bloqueado debe devolverse.
- ❖ Se debe crear wallet_transaction tipo refund.
- ❖ Debe emitirse BetStatusUpdated y WalletUpdated.

Prioridad: **MEDIA**

HU-37: Rechazar apuesta inválida

Como sistema,
quiero rechazar apuestas inválidas,
para evitar inconsistencias y riesgos financieros.

Criterios de aceptación

- ❖ Debe rechazarse si el evento está cerrado.
- ❖ Debe rechazarse si la cuota está vencida.
- ❖ Debe rechazarse si el usuario no tiene saldo.
- ❖ Debe rechazarse si el usuario está bloqueado.
- ❖ Debe registrarse el motivo del rechazo.

Prioridad: **ALTA**

HU-38: Crear apuestas combinadas

Como usuario,
quiero seleccionar varias cuotas en un solo ticket,
para realizar una apuesta combinada.

Criterios de aceptación

- ❖ El usuario puede seleccionar más de un evento.
- ❖ El sistema debe calcular cuota total multiplicando las cuotas.
- ❖ Todas las selecciones deben estar vigentes.
- ❖ Si una selección cambia, debe pedirse confirmación.
- ❖ La apuesta combinada solo gana si todas sus selecciones ganan.

Prioridad

Prioridad: **ALTA**

Módulo 7: Liquidación de apuestas

HU-39: Consultar resultados de eventos

Como sistema,
quiero consultar resultados o estados finales de eventos,
para determinar si una apuesta ganó o perdió.

Criterios de aceptación

- ❖ El sistema debe obtener resultados del proveedor o permitir carga manual.
- ❖ El resultado debe asociarse al evento correcto.
- ❖ Los eventos finalizados deben marcarse como completed.

Prioridad: **ALTA**

HU-40: Liquidar apuesta ganadora

Como sistema,
quiero liquidar una apuesta ganadora,
para acreditar la ganancia al usuario.

Criterios de aceptación

- ❖ La apuesta debe pasar a estado won.
- ❖ Se debe liberar el saldo bloqueado.
- ❖ Se debe acreditar la ganancia correspondiente.
- ❖ Se debe crear wallet_transaction tipo bet_win.
- ❖ Se debe notificar al usuario en tiempo real.

Prioridad: **ALTA**

HU-41: Liquidar apuesta perdedora

Como sistema,
quiero liquidar una apuesta perdedora,
para descontar definitivamente el monto apostado.

Criterios de aceptación

- ❖ La apuesta debe pasar a estado lost.
- ❖ El locked_balance debe reducirse.
- ❖ Se debe crear wallet_transaction tipo bet_loss.
- ❖ Se debe notificar al usuario en tiempo real.

Prioridad: **ALTA**

HU-42: Reembolsar apuesta cancelada

Como sistema,
quiero reembolsar apuestas de eventos cancelados,
para devolver el saldo al usuario.

Criterios de aceptación

- ❖ La apuesta debe pasar a estado refunded.
- ❖ El monto apostado debe volver al balance.
- ❖ El locked_balance debe reducirse.
- ❖ Se debe crear wallet_transaction tipo refund.
- ❖ Se debe notificar al usuario.

Prioridad: **ALTA**

HU-43: Liquidación manual por administrador

Como administrador,
quiero liquidar una apuesta manualmente,
para resolver casos donde el resultado automático no esté disponible.

Criterios de aceptación

- Solo el administrador puede liquidar manualmente.
- Debe seleccionar resultado: ganada, perdida o reembolsada.
- Debe ingresar una observación.
- El sistema debe guardar auditoría.
- Debe actualizar wallet y estado de apuesta.

Prioridad: **MEDIA**

Módulo 8: Administración

HU-44: Dashboard administrativo

Como administrador,
quiero ver un dashboard general,
para monitorear usuarios, apuestas, saldo y consumo de API.

Criterios de aceptación

- ❖ Debe mostrar total de usuarios.
- ❖ Debe mostrar apuestas pendientes.
- ❖ Debe mostrar monto total apostado.
- ❖ Debe mostrar ganancias/pérdidas globales.
- ❖ Debe mostrar consumo de The Odds API.

Prioridad: **ALTA**

HU-45: Listar apuestas en administración

Como administrador,
quiero listar todas las apuestas,
para revisar la actividad de los usuarios.

Criterios de aceptación

- ❖ Debe estar paginado.
- ❖ Debe permitir filtrar por usuario, estado y fecha.
- ❖ Debe mostrar monto, cuota, ganancia potencial y estado.
- ❖ Debe permitir ver detalle.

Prioridad: **ALTA**

HU-46: Ver detalle administrativo de apuesta

Como administrador,
quiero ver el detalle completo de una apuesta,
para auditar su creación, cuota usada y transacciones.

Criterios de aceptación

- ❖ Debe mostrar usuario.
- ❖ Debe mostrar evento y selección.
- ❖ Debe mostrar snapshot de cuota.
- ❖ Debe mostrar movimientos de wallet.
- ❖ Debe mostrar historial de cambios de estado.

Prioridad: **MEDIA**

HU-47: Gestionar configuración del sistema

Como administrador,
quiero modificar parámetros del sistema,
para controlar reglas de apuestas y sincronización.

Criterios de aceptación

- ❖ Puede modificar monto mínimo de apuesta.
- ❖ Puede modificar monto máximo de apuesta.
- ❖ Puede modificar región por defecto.
- ❖ Puede modificar mercado por defecto.
- ❖ Puede modificar intervalo de sincronización.

Prioridad: **ALTA**

HU-48: Ver logs de proveedor externo

Como administrador,
quiero ver logs de The Odds API,
para auditar consumo, errores y respuestas del proveedor.

Criterios de aceptación

- ❖ Debe mostrar endpoint consultado.
- ❖ Debe mostrar deporte, región y mercado.
- ❖ Debe mostrar créditos usados.
- ❖ Debe mostrar estado de respuesta.
- ❖ Debe mostrar fecha y hora.

Prioridad: **MEDIA**

HU-49: Gestionar usuarios desde administración

Como administrador,
quiero listar y gestionar usuarios,
para controlar el acceso a la plataforma.

Criterios de aceptación

- ❖ Debe listar usuarios paginados.
- ❖ Debe permitir filtrar por estado.
- ❖ Debe permitir bloquear o desbloquear usuarios.
- ❖ Debe mostrar saldo del usuario.

Prioridad: **MEDIA**

Módulo 9: Auditoría y seguridad

HU-50: Registrar auditoría de operaciones críticas

Como sistema,
quiero registrar operaciones críticas,
para tener trazabilidad de acciones sensibles.

Criterios de aceptación

- ❖ Debe auditar creación de apuestas.
- ❖ Debe auditar cambios de wallet.
- ❖ Debe auditar liquidaciones manuales.
- ❖ Debe auditar cambios de configuración.
- ❖ Debe registrar usuario, acción, fecha y datos relevantes.

Prioridad: **ALTA**

HU-51: Proteger endpoints por roles

Como sistema,
quiero proteger endpoints según roles,
para impedir accesos no autorizados.

Criterios de aceptación

- ❖ Endpoints de usuario requieren autenticación.
- ❖ Endpoints admin requieren rol admin.
- ❖ Endpoints operator requieren permisos específicos.
- ❖ Si el usuario no tiene permiso, debe devolver 403.

Prioridad: **ALTA**

HU-52: Evitar apuestas concurrentes con el mismo saldo

Como sistema,
quiero controlar concurrencia al crear apuestas,
para evitar que un usuario use el mismo saldo varias veces.

Criterios de aceptación

- ❖ La creación de apuesta debe usar transacción de base de datos.
- ❖ La wallet debe bloquearse durante la operación.
- ❖ Si dos apuestas ocurren al mismo tiempo, solo debe aceptarse si hay saldo suficiente real.

- ❖ No debe quedar saldo negativo.

Prioridad: **ALTA**

HU-53: Evitar sincronizaciones duplicadas

Como sistema,
quiero evitar que dos jobs sincronicen las mismas cuotas al mismo tiempo,
para no duplicar datos ni gastar créditos innecesarios.

Criterios de aceptación

- ❖ El sistema debe usar locks de cache.
- ❖ Si ya existe una sincronización activa para un deporte, debe evitar otra simultánea.
- ❖ Debe liberar el lock al finalizar.
- ❖ Debe registrar errores si el job falla.

Prioridad: **ALTA**

Módulo 10: Notificaciones

HU-54: Notificar creación de apuesta

Como usuario,
quiero recibir confirmación cuando mi apuesta se crea correctamente,
para saber que fue registrada.

Criterios de aceptación

- ❖ El sistema debe devolver respuesta exitosa al crear apuesta.
- ❖ Debe emitir evento realtime.
- ❖ El frontend debe mostrar mensaje de confirmación.

Prioridad: **ALTA**

HU-55: Notificar apuesta rechazada

Como usuario,
quiero recibir una explicación cuando una apuesta sea rechazada,
para entender qué ocurrió.

Criterios de aceptación

- ❖ El sistema debe devolver motivo del rechazo.
- ❖ Motivos posibles: saldo insuficiente, evento cerrado, cuota vencida, usuario bloqueado.
- ❖ El frontend debe mostrar el mensaje claramente.

Prioridad: **ALTA**

HU-56: Notificar resultado de apuesta

Como usuario,
quiero recibir una notificación cuando mi apuesta sea liquidada,
para saber si gané, perdí o recibí reembolso.

Criterios de aceptación

- ❖ El sistema debe emitir evento cuando cambie el estado de la apuesta.
- ❖ Debe actualizar el historial.
- ❖ Debe actualizar la wallet.
- ❖ Debe mostrar mensaje de resultado.

Prioridad: **ALTA**

Módulo 11: Reportes

HU-57: Reporte de apuestas por fecha

Como administrador,
quiero generar reportes de apuestas por rango de fechas,
para analizar la actividad de la plataforma.

Criterios de aceptación

- ❖ Debe permitir seleccionar fecha inicio y fecha fin.
- ❖ Debe mostrar total apostado.
- ❖ Debe mostrar total ganado por usuarios.
- ❖ Debe mostrar total perdido por usuarios.
- ❖ Debe permitir filtrar por estado.

Prioridad: **MEDIA**

HU-58: Reporte de movimientos de wallet

Como administrador,
quiero ver movimientos de wallet,
para auditar depósitos, retiros, apuestas y pagos.

Criterios de aceptación

- ❖ Debe listar transacciones paginadas.
- ❖ Debe permitir filtrar por usuario, tipo y fecha.
- ❖ Debe mostrar saldo antes y después.
- ❖ Debe mostrar referencia de la operación.

Prioridad: **MEDIA**

HU-59: Reporte de consumo de API externa

Como administrador,
quiero generar un reporte de consumo de The Odds API,
para controlar costos y créditos.

Criterios de aceptación

- ❖ Debe mostrar llamadas por día.
- ❖ Debe mostrar créditos usados.
- ❖ Debe mostrar endpoints más consumidos.
- ❖ Debe mostrar errores del proveedor.

Prioridad: **MEDIA**

Módulo 12: Configuración futura

HU-60: Soporte para varios mercados de apuesta

Como administrador,
quiero habilitar varios mercados de apuesta,
para ofrecer más opciones a los usuarios.

Criterios de aceptación

- ❖ El sistema debe soportar h2h.
- ❖ El sistema debe soportar spreads.
- ❖ El sistema debe soportar totals.
- ❖ El sistema debe permitir activar o desactivar mercados.

Prioridad: **FUTURA**

HU-61: Soporte para múltiples regiones

Como administrador,
quiero habilitar varias regiones de bookmakers,
para ampliar la cobertura de cuotas.

Criterios de aceptación

- ❖ El sistema debe permitir configurar regiones.
- ❖ Debe soportar eu, us, uk, au según proveedor.
- ❖ El consumo de créditos debe calcularse considerando regiones activas.

Prioridad: **FUTURA**

HU-62: Soporte para plan pagado de The Odds API

Como administrador,
quiero adaptar la frecuencia de actualización cuando se adquiere un plan pagado,
para mejorar la experiencia en tiempo real.

Criterios de aceptación

- ❖ La frecuencia de sincronización debe ser configurable.
- ❖ Se deben poder habilitar más deportes.
- ❖ Se deben poder habilitar más mercados.
- ❖ No debe requerir cambios grandes en arquitectura.

Prioridad: **FUTURA**

BASE DE DATOS

Este es el link: <https://dbdiagram.io/d/6a039b1354a51d93d30cd709>